Learning Objectives

By the end of this session, you should be able to:

- Extend a recursive-descent parser to support `return` statements, function parameters, and variable declarations.
- Connect expression parsing and code generation to produce executable programs with return values.
- Use symbol tables to manage variable names, scopes, and storage locations.
- Extend expression trees to represent variable references and assignments.
- Distinguish between computing the value of an expression and computing the address of a variable.
- Generate code for reading and writing variables.
- Combine these features to compile programs that use global variables, local variables, and function parameters.

Step 1 — Our First Return Statement

In Worksheet 19, we extended our code generator with `genReturn()`. However, our parser does not yet support `return` statements. Let's change that!

Consider `examples/ws20_return.not-abc`:

```
fn main()
{
    return 42;
}
```

Our goal is to compile and execute this program. If everything works, it should terminate with exit code `42`.

Tasks

1. **Extend the grammar.** We introduce a new rule for return statements and allow them inside compound statements:

   ```
   top-level     = { fn } EOI
   fn            = "fn" identifier "(" ")" compound-stmt
   compound-stmt = "{" { expr-stmt | return-stmt } "}"
   expr-stmt     = expr ";"
   return-stmt   = "return" expr ";"
   ```

   Notice that a return statement consists of the keyword `return`, followed by an expression and a semicolon.

2. **Implement `parseReturnStmt()`.** In `parser.abc`, add the following function at the end of the file. Remember to add a forward declaration near the beginning, alongside the other parser function declarations.

   ```
   fn parseReturnStmt(): bool
   {
       if (token.kind != RETURN) {
           return false;
       }
       getToken();
       local expr: -> Expr = parseExpr();
       if (!expr) {
           fprintf(stderr, "expected expression\n");
           exit(1);
           return false;
       }
       local retVal: GenReg = loadExpr(expr);
       genReturn(retVal);
       genReleaseReg(retVal);
       releaseExpr(expr);
       expected(SEMICOLON);
       getToken();
       return true;
   }
   ```

   Notice how the parser connects the components we have already implemented: `parseExpr()` constructs an expression tree, `loadExpr()` generates code to evaluate it, and `genReturn()` generates the instructions needed to return the result. We then release the register and expression tree.

© Michael C. Lehn | CC BY-SA 4.0

3. **Update `parseCompoundStmt()`.** A compound statement may now contain both expression statements and return statements. Replace its implementation with:

```
fn parseCompoundStmt(): bool
{
    if (token.kind != LBRACE) {
        return false;
    }
    getToken();
    while (parseExprStmt() || parseReturnStmt())
    {
    }
    expected(RBRACE);
    getToken();
    return true;
}
```

   Notice how closely the while loop follows our grammar. Thanks to short-circuit evaluation of `||`, we first try to parse an expression statement. If that fails, we try a `return` statement.

4. **Build and test.** After `make` generate, assemble, and execute the program:

```
./not-abc < examples/ws20_return.not-abc > foo.s
ulmas foo.s
ulm a.out
echo $?
```

   The expected output is: `42`. Congratulations! Our compiler can now compile a function containing a `return` statement and generate an executable program. Take a look at `foo.s` as well and identify the instructions generated by `genReturn()` and the shared function epilogue?

Step 2 — Making Error Messages Useful

Error handling is tedious. But what is even worse than writing error-handling code? Getting error messages that don't tell us where something went wrong! For example, what happens if we forget the expression after `return`?

```
fn main()
{
    return;
}
```

Our parser detects the error, but the message expected expression does not tell us where the expression was expected. We should at least report the line and column. Unfortunately, our existing `expected()` function only works when we expect a particular token kind. Here, we expect an entire expression.

Tasks

1. Introduce a common error-reporting function. In `parseexpr.hdr`, declare:

```
extern fn error(msg: ->readonly char): bool;
```

   Implement it in `parseexpr.abc`:

```
fn error(msg: ->readonly char): bool
{
    fprintf(stderr, "%u.%u: error: %s\n", token.pos.row, token.pos.col, msg);
    exit(1);
    return false;
}
```

   Notice that `error()` reports the position of the current token. It also returns `bool`, even though `exit(1)` terminates the program before the `return` statement can be reached.

2. **Use it in `parseReturnStmt()`.** Replace the previous error handling with

```
if (!expr) {
    error("expected expression");
    return false;
}
```

   Rebuild the compiler and try compiling the invalid program above. Does the error message now identify the position where the expression was expected?

3. **Clean up our parser.** Search `parser.abc` and `parseexpr.abc` for other places where we still use combinations of `fprintf(stderr, ...)` and `exit(1)` to report errors. Replace them with calls to `error()` wherever appropriate.

© Michael C. Lehn | CC BY-SA 4.0

Step 3 — Functions with Parameters

So far, our compiler only supports function definitions without parameters. Let's change that! First, extend our grammar to allow any number of parameters:

```
fn = "fn" identifier "(" [ identifier { "," identifier } ] ")" compound-stmt
```

Consider `examples/ws20_diff.not-abc`:

```
fn diff(a, b)
{
    return 42;
}

fn main()
{
    return diff(2 * (7 + 3), 12 + 5 * 2);
}
```

Admittedly, `diff` does not make much use of its parameters yet. We'll fix that soon! For now, our goal is to register the parameters correctly and generate an executable program.

Tasks

Extend the parser. In `parser.abc`, update `parseFn()` by inserting the following code at the appropriate position:

```
    while (token.kind == IDENTIFIER) {
        genFuncDefParam(token.val) || error("duplicate parameter name");
        getToken();
        if (token.kind != COMMA) {
            break;
        }
        getToken();
        expected(IDENTIFIER);
    }
```

Notice how the loop corresponds to the optional parameter list in our grammar. Each parameter is passed to `genFuncDefParam()`, which handles its entry in the symbol table. Also notice how short-circuit evaluation of `||` lets us report duplicate parameter names using the `error()` function introduced in Step 2.

Rebuild everything using `make`, compile the example, and execute it. Check that you get the expected exit code `42`.

© Michael C. Lehn | CC BY-SA 4.0

Step 4 — Loading and Storing Variables (Preparation)

Things are getting more interesting! Our code generator can already register variable names in the symbol table. But it cannot yet access their values. We need to distinguish between a variable's address and the value stored at that address. We also need to support global variables, which live outside function stack frames.

Tasks

1. **Extend our test program.** Update `xtest_gen_ex03.abc` as follows:

```
@ "gen.hdr"
@ "symtab.hdr"

fn main()
{
    genInit();

    // global x: i64;
    genAddGlobal("x");

    // fn diff(a, b) {
    genFuncDefBegin("diff");
    genFuncDefParam("a");
    genFuncDefParam("b");

    // x = a;
    local a: GenReg = genLoadVar("a");
    local xAddr: GenReg = genLoadAddr("x");
    genStore(a, xAddr);
    genReleaseReg(a);
    genReleaseReg(xAddr);

    // local r: i64;
    genAddLocal("r");

    // r = b;
    local bAddr: GenReg = genLoadAddr("b");
    local b: GenReg = genFetch(bAddr);
    genReleaseReg(bAddr);
    local rAddr: GenReg = genLoadAddr("r");
    genStore(b, rAddr);
    genReleaseReg(b);
    genReleaseReg(rAddr);

    // return x - r;
    local result: GenReg = genInstr(GEN_SUB,
                                    genLoadVar("x"),
                                    genLoadVar("r"));
    genReturn(result);
    genReleaseReg(result);

    // }
    genFuncDefEnd();

    // fn main() { return diff(72, 30); }
    genFuncDefBegin("main");
    genCallBegin("diff", 2);
    genCallAddArg(genConstant("72"), 0);
    genCallAddArg(genConstant("30"), 1);
    local retVal: GenReg = genCallEnd();
    genReturn(retVal);
    genReleaseReg(retVal);
    genFuncDefEnd();
}
```

   Before continuing, write down the equivalent program in ABC. Can you identify where each of the new code generator functions is used?

2. **Extend the interface.** Add these declarations to `gen.hdr`:

```
extern fn genAddGlobal(ident: ->readonly char): bool;

extern fn genLoadAddr(ident: ->readonly char): GenReg;
extern fn genLoadVar(ident: ->readonly char): GenReg;
extern fn genFetch(addr: GenReg): GenReg;
extern fn genStore(val: GenReg, addr: GenReg);
```

   Add stub implementations to `gen.abc`. and rebuild using `make`.

© Michael C. Lehn | CC BY-SA 4.0

Step 5 — Loading and Storing Variables (Implementation)

Now let's implement the functions introduced in Step 4. We will start with global variables, then compute variable addresses, and finally generate instructions to read and write memory.

Tasks

1. **Implement `genAddGlobal()`.** Global variables are stored in the BSS segment. Implement:

```
fn genAddGlobal(ident: ->readonly char): bool
{
    if (symtabAddGlobal(ident)) {
        setSegment(BSS);
        printf("\t.align 8\n");
        printf("%s:\n", ident);
        printf("\t.quad\n");
        return true;
    } else {
        return false;
    }
}
```

   The function registers the identifier in the global symbol table, switches to the BSS segment, aligns the variable to an 8-byte boundary, and reserves space for it. If the identifier has already been defined, the function returns `false` without generating assembly code.

2. **Implement `genLoadAddr()`** to generate code that computes the address of a variable and stores it in a register:

```
fn genLoadAddr(ident: ->readonly char): GenReg
{
    local dest: GenReg = getReg();
    local info: SymtabInfo;
    if (!symtabGetInfo(ident, &info)) {
        assert(0 && "symtabGetInfo(ident, &info) failed");
    }
    setSegment(TEXT);
    if (info.isGlobal) {
        printf("\tloadz \t%s, \t%%%d\n", ident, dest);
    } else {
        if (info.offset > 0) {
            printf("\taddq \t%d, \t%%2, \t%%%d\n", info.offset, dest);
        } else {
            printf("\tsubq \t%d, \t%%2, \t%%%d\n", -info.offset, dest);
        }
    }
    return dest;
}
```

   Notice the difference:

   1. **Global variables** have addresses represented by assembly labels. The assembler resolves these addresses.
   2. **Parameters and local variables** are stored in stack frames. Their offsets relative to the frame pointer `%2` are retrieved from the symbol table.

   Our ISA only supports unsigned immediate operands for `addq` and `subq`. Therefore, we use `addq` for positive offsets and `subq` for negative offsets.

   We assume the parser checks variable declarations, so a failed `symtabGetInfo()` indicates an internal compiler error.

3. **Implement `genFetch()` and `genStore()`.** These two functions provide the basic operations for accessing memory through an address stored in a register:

```
fn genFetch(addr: GenReg): GenReg
{
    local dest: GenReg = getReg();
    setSegment(TEXT);
    printf("\tmovq \t(%%%d), \t%%%d\n", addr, dest);
    return dest;
}

fn genStore(val: GenReg, addr: GenReg)
{
    setSegment(TEXT);
    printf("\tmovq \t%%%d, \t(%%%d)\n", val, addr);
}
```

© Michael C. Lehn | CC BY-SA 4.0

4. **Implement `genLoadVar()`.** Finally, we can combine address calculation and memory access:

```
fn genLoadVar(ident: ->readonly char): GenReg
{
    local dest: GenReg = genLoadAddr(ident);
    genReleaseReg(dest);
    return genFetch(dest);
}
```

   Notice the small register-allocation trick: We release the address register before we are calling `genFetch()`. This allows `getReg()` to reuse that register. Releasing a register does not erase its contents, so the generated code can still use the address until the register is overwritten.
5. **Test everything.** Rebuild using make and run `xtest_gen_ex03`. Assemble and execute the generated program. If everything works, the exit code should be `42`.
   Inspect the generated assembly code as well. Can you identify the instructions that compute addresses, fetch values, and store values?
6. **Hunting Register Leaks.** Until now, the last computed value often ended up in register `%4`. With our extended code generator, we can no longer rely on that.
   Take another look at `xtest_gen_ex03.abc`. Can you find places where registers are allocated but never released? These are register leaks: registers that are no longer needed but remain marked as being in use.
   For our small test program, this may not cause an immediate problem. However, if the compiler keeps running out of available registers because it fails to release them, it will eventually be unable to generate correct assembly code.
   Your task: Find and fix the register leaks in `xtest_gen_ex03.abc` using `genReleaseReg()`. Make sure that registers are released only after their values are no longer needed. Rebuild and verify that the program still returns `42`.

Step 6 — Adding Identifier Expressions

In Step 5, we taught our code generator how to access variables. But our expression trees still cannot represent variable references. Let's fix that!
As usual, we'll start with a test program. Create `xtest_expr_var.abc`:

```
@ <stdio.hdr>
@ "expr.hdr"
@ "gen.hdr"

fn main()
{
    local expr: -> Expr;

    expr = createIdentifierExpr(UStrCreate("x"));
    genAddGlobal(expr->primary.identifier);

    printf("\t# load expression: ");
    printExprTree(expr);
    printf("\n");
    genReleaseReg(loadExpr(expr));
    releaseExpr(expr);
}
```

Before implementing anything, take another look at `expr.hdr`. Back in Session 16, we already sneaked an `identifier` member into the Primary structure of `Expr`:

```
struct Expr
{
    kind: ExprKind;
    union {
        // ...
        primary: struct Primary {
            decimal: ->UStr;
            identifier: ->UStr;
            // ...
        };
        // ...
    };
};
```

So we don't need to change this structure. But there are still a few pieces missing.

© Michael C. Lehn | CC BY-SA 4.0

Tasks

1. **Extend ExprKind and the interface.** Add `EXPR_IDENTIFIER` to the group of primary expressions in `ExprKind`. Also declare the constructor in `expr.hdr`:

   ```
   extern fn createIdentifierExpr(val: ->UStr): ->Expr;
   ```

   Run `make`. If everything is correct so far, you should now get *only a linker error*: the function has been declared and used, but not yet implemented.
2. **Implement the constructor.** This is almost identical to `createIntegerExpr()`. We just need to set the appropriate expression kind and initialize `primary.identifier`:

   ```
   fn createIdentifierExpr(val: ->UStr): ->Expr
   {
       local n: -> Expr = malloc(sizeof(*n));
       assert(n);
       n->kind = EXPR_IDENTIFIER;
       n->primary.identifier = val;
       return n;
   }
   ```

   Run `make` again. The test program should now compile and link, but assertion failures will reveal what is still missing. ;-) Insert these puzzle pieces at the right places:

   ```
   case EXPR_IDENTIFIER:
       printf("[%s]", expr->primary.identifier);
       break;
   ```

   and

   ```
   case EXPR_IDENTIFIER: return genLoadVar(expr->primary.identifier);
   ```
3. Explain to your neighbor—or, even better, to us—why `releaseExpr()` does not need to be modified to handle `EXPR_IDENTIFIER` nodes.

Step 7 — Identifiers in Expressions

Let's finally put our function parameters to work! Modify `examples/ws20_diff.not-abc`:

```
fn diff(a, b)
{
    return a - b;
}

fn main()
{
    return diff(8 * 8 + 10 - 2, 2 * (7 +8));
}
```

If everything works, our program should terminate with exit code `42`.

Tasks

1. **Try compiling the program.** Run `make` first if necessary, then:

   ./not-abc < examples/ws20_diff.not-abc > examples/ws20_diff.s

   Unfortunately, our compiler reports an error. The reason is that our expression parser still implements this grammar rule for a factor (also called a primary expression):

   ```
   factor = decimal-literal
          | character-literal
          | identifier "(" [ expr { "," expr } ] “)"
          | "(" expr ")"
   ```

   An identifier is only allowed as the name of a function being called. Let's change that!
2. **Extend the grammar and parser.** Our new grammar rule is:

   ```
   factor = decimal-literal
          | character-literal
          | identifier [ "(" [ expr { "," expr } ] ")" ]
          | "(" expr ")"
   ```

   After reading an identifier, `parseFactor()` must check whether the next token is `(`. If so, it creates a function call expression. Otherwise, it creates an identifier expression.

   Fortunately, this requires only a small modification. Replace the existing code

© Michael C. Lehn | CC BY-SA 4.0

```
    } else if (token.kind == IDENTIFIER) {
        local ident: ->UStr = token.val;
        getToken();
        expected(LPAREN);
        // ... stuff ...
        return callExpr;
```

with

```
    } else if (token.kind == IDENTIFIER) {
        local ident: ->UStr = token.val;
        getToken();
        if (token.kind == LPAREN) {
            // ... stuff as before ...
            return callExpr;
        } else {
            symtabIsDefined(ident) || error("undefined identifier");
            return createIdentifierExpr(ident);
        }
```

Keep the existing code for parsing function call arguments inside the `if` branch.

3. **Build and test.** Rebuild the compiler, translate `examples/ws20_diff.not-abc`, assemble the generated code, and run it. Does the program now return exit code `42`?

© Michael C. Lehn | CC BY-SA 4.0